

detection. However, the NOVID app has Wi-Fi and Bluetooth capabilities as well.

BLE in DCT systems

While there are examples of Covid-19 apps that utilise Wi-Fi and ultrasound technologies, a large majority of these systems use Bluetooth as the underlying technology of communication. Within Bluetooth also, there is a low energy variant, called Bluetooth Low Energy (BLE), that has caught the attention of most of the system designers and implementers.

Bluetooth Special Interest Group (Bluetooth SIG) originally proposed BLE as a wireless personal area network technology for “novel applications in the healthcare, fitness, beacons, security, and home entertainment industries.” BLE is natively supported by a host of mobile operating systems including iOS, Android, and Windows Phone. It consumes much less power than the classic Bluetooth, operates in the same spectrum range, and both the protocols can co-exist in a mobile device.

The possibility of using Bluetooth communication for contact tracing during an epidemic was shown as feasible through detailed simulations by a group of researchers, and the results were published way back in 2014. During the current pandemic, this idea was adapted and expanded by multiple groups of experts and scientists from different countries, and most of these protocol designs considered BLE as the underlying technology for device-to-device communication.

It is important to understand that the BLE layer does not have any provision to encrypt packets. So, packets exchanged between devices through BLE are transparent and anyone may eavesdrop and observe the data contained in their payloads. However, in case of Covid-19 apps this fact does not pose any direct threat to privacy or security. Let us understand why it is so.

We have already mentioned earlier that the DCT apps share constantly

Time	BLE MAC	Proximity Identifier
00:00:00	BLE_MAC_1	RPI_1
00:05:00	BLE_MAC_2	RPI_1
00:10:00	BLE_MAC_2	RPI_2
00:15:00	BLE_MAC_3	RPI_2

Table 1: Intervals of Proximity Identifier

rotating pseudo-random identifiers when any two devices come in proximity to each other. So when Alice and Bob meet for a brief period of fifteen minutes in a park, Alice’s device (say A) emits a pseudo-random number (say RPI_1) for initial ten minutes, and then another number (say RPI_2) for the remaining five minutes. Bob’s device (say B) detects packets containing these numbers and stores them in B’s local storage. Similarly, A also captures the numbers shared by B during this time-window.

Even if an eavesdropper (say Eve) intercepts some of the packets and stores such data, she would not be able to determine any personal or Personally Identifiable Information (PII) of Alice or Bob solely through these packets. That is why the unencrypted nature of a BLE packet does not pose any serious complication. However, there could still be a possibility of a linkage attack.

If nothing can be determined from any of these pseudo-random identifiers, then one may ask how these data packets may eventually be utilised for the purpose of contact tracing?

Broadly speaking, in every DCT system, there is a mechanism to store the secret keys, or some variants of those keys, in either the local storage of the devices or in a back-end server. When a user (say Bob) is diagnosed positive, the keys, or the sent/received pseudo-random identifiers of his device (B), are analysed to determine the possible matches with the keys or the sent/received identifiers of another user’s (say Alice’s) device (A). The fact that these

identifiers can be regenerated anytime from their originating keys makes them pseudonymous instead of anonymous. Hence, the system has a mechanism to determine the close encounters through the usage of secret keys, even though the pseudo-random identifiers on their own may appear gibberish to Eve, the eavesdropper.

The communication of BLE packets may take place either in a broadcast topology mode or in a connected topology mode.

In broadcast mode, a device alternates between two separate roles - the broadcaster role and the observer role. In the broadcaster role, the device periodically sends advertising packets that can be received by any other device. Similarly, in the observer role the device periodically scans (in a pre-set frequency) to receive advertised packets from other devices. During broadcasting, one device may send packets to multiple other devices and it works in a non-connectable fashion.

In broadcast mode, the payload size is restricted to at most 31 bytes. There is an option to send a larger payload (called Scan Response) as well through which this size can be extended up to a maximum of 62 bytes. When a DCT protocol uses broadcast mode, the app would have no clue during transmission of proximity identifiers about whether any other nearby device/app is receiving the packets or not.

In connected topology mode, the device-to-device communication takes place in pairwise fashion, where one device plays the role of Central (or master) and the other plays the role of Peripheral (or slave). A device alternates between the two roles. The connection is established by the device playing the role of Central when it detects packets advertised by another device running in Peripheral mode. A Peripheral can connect with multiple Centrals at a time, and a Central can also connect with multiple Peripherals. In this mode of communication, a device always knows